04 - Arreglos

Pregunta número 1

Which of the following are valid ways to initialize an array in Java?

Seleccione una o más de una:

- [ ] a. int[] arr = (1, 2, 3);
- [x] b. int[] arr = {1, 2, 3};
- [x] c. int[] arr = new int[] {1, 2, 3};
- [x] d. int[] arr = new int[3];

Respuesta: El inciso a es incorrecto porque Java utiliza llaves {} y no paréntesis () para delimitar los valores en la inicialización de un arreglo. El inciso b es válido, ya que es la forma abreviada de inicialización, permitida exclusivamente en la misma instrucción donde se declara la variable. El inciso c también es válido, ya que representa la instanciación explícita de un arreglo anónimo indicando sus valores, sin especificar el tamaño entre los corchetes (ya que el compilador lo infiere por la cantidad de elementos). Finalmente, el inciso d es válido, ya que instancia un arreglo de tamaño 3 en memoria, inicializando automáticamente cada elemento con el valor por defecto para el tipo primitivo int, que es 0.

Pregunta número 2

¿Qué imprime este código de matriz?

```
int array[][] = {
    {1,2,3},
    {4,5,6},
    {7,8,9}
};
int result = 0;
for(int i = 0; i < array.length; i++) {
    result += array[i][i];
}
System.out.println(result);
```

Seleccione una:

- ( ) a. 0
- ( ) b. 9
- ( ) c. 18
- (•) d. 15

Respuesta: En este código se están sumando los valores que conforman la diagonal principal de la matriz bidimensional. Hay que desglosar el ciclo iteración por iteración para entender qué está pasando: Primero, se inicializa la variable result en 0. El ciclo for se ejecuta 3 veces porque array.length (el número de filas de la matriz principal) es 3. La clave de este ejercicio se encuentra en la instrucción result += array[i][i], la cual utiliza el mismo índice para la fila y la columna. Lo siguiente que sucede en la iteración 1 (i=0) es que se accede a array[0][0], que es 1. Lo suma a result, quedando en 1. En la iteración 2 (i=1) se accede a array[1][1], que es 5. Lo suma a result, quedando en 1 + 5 = 6. En la tercera iteración 3 (i=2) se accede a array[2][2], que es 9. Lo suma a result, quedando en 6 + 9 = 15. Finalmente, el ciclo termina y se imprime el valor final acumulado de 15.

Pregunta número 3

What is the result?

```
public class NestedForLoopTest {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };
        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[i].length; j++) {
                if (i == j) {
                    System.out.print(matrix[i][j] + " ");
                }
            }
        }
    }
}
```

Seleccione una:

- a. 147
- b. 123456789
- c. 159
- d. 159246837

Respuesta: En este ejercicio tenemos exactamente el mismo propósito lógico que en nuestro ejercicio anterior, o sea, extraer la diagonal principal de la matriz, pero la implementación se realiza utilizando una estructura diferente con bucles anidados. Hay que desglosar su comportamiento para entender mejor qué está pasando: Primero tenemos el bucle externo `for` con la variable i que se encarga de recorrer las filas de la matriz (índices 0, 1, 2). El bucle interno `for` con la variable j recorre cada una de las columnas dentro de la fila actual. La instrucción clave es la condición if (i == j). Esto actúa como un filtro que ignora la mayoría de los elementos de la matriz y solo permite ejecutar el System.out.print cuando el índice de la fila es exactamente igual al índice de la columna. Siguiendo esta regla, tenemos que cuando i = 0 y j = 0, se imprime matrix[0][0], que es 1. Cuando i = 1 y j = 1, se imprime matrix[1][1], que es 5. Cuando i = 2 y j = 2, se imprime matrix[2][2], que es 9. La salida del programa es justamente la concatenación de los números impresos en la diagonal principal, o sea, 1, 5 y 9.

Pregunta número 4

What is the result?

```
public class Test {
    public static void main(String[] args){
        int[][] array = {{0},{0,1},{0,2,4},{0,3,6,9},{0,4,8,12,16}};
        System.out.println(array[4][1]);
        System.out.println(array[1][4]);
    }
}
```

Seleccione una:

- a. 4 Null
- b. Null 4.
- c. 4 An ArrayIndexOutOfBoundsException is thrown at run time.
- d. An IllegalArgumentException is thrown at run time

Respuesta: En este ejercicio se está evaluando el manejo de arreglos irregulares (jagged arrays) en Java, donde cada fila de una matriz bidimensional puede tener una longitud distinta. Veamos el flujo de ejecución para poder entender lo que está pasando. Aquí en la primera instrucción (System.out.println(array[4][1]);) tenemos que el código primero accede a la fila con el índice 4 (la quinta fila del arreglo principal), la cual corresponde al arreglo {0, 4, 8, 12, 16}. Dentro de esa fila, solicita el elemento en el índice 1 (el segundo elemento). Esto es completamente válido y da como resultado la impresión del número 4. En la segunda instrucción (System.out.println(array[1][4])) tenemos que el código accede a la fila con el índice

1 (la segunda fila), correspondiente al arreglo {0, 1}. Este arreglo interno tiene una longitud de tan solo 2 elementos, por lo que sus únicos índices válidos son el 0 y el 1. Al intentar acceder al índice 4, el cual sobrepasa la longitud del arreglo, la JVM detiene la ejecución inmediatamente lanzando una excepción ArrayIndexOutOfBoundsException.

Pregunta número 5

What is the result?

```
public static void main(String[] args) {
    int[][] array2D = {{0,1,2}, {3,4,5,6}};
    System.out.print(array2D[0].length +"");
    System.out.print(array2D[1].getClass().isArray() +"");
    System.out.println(array2D[0][1]);
}
```

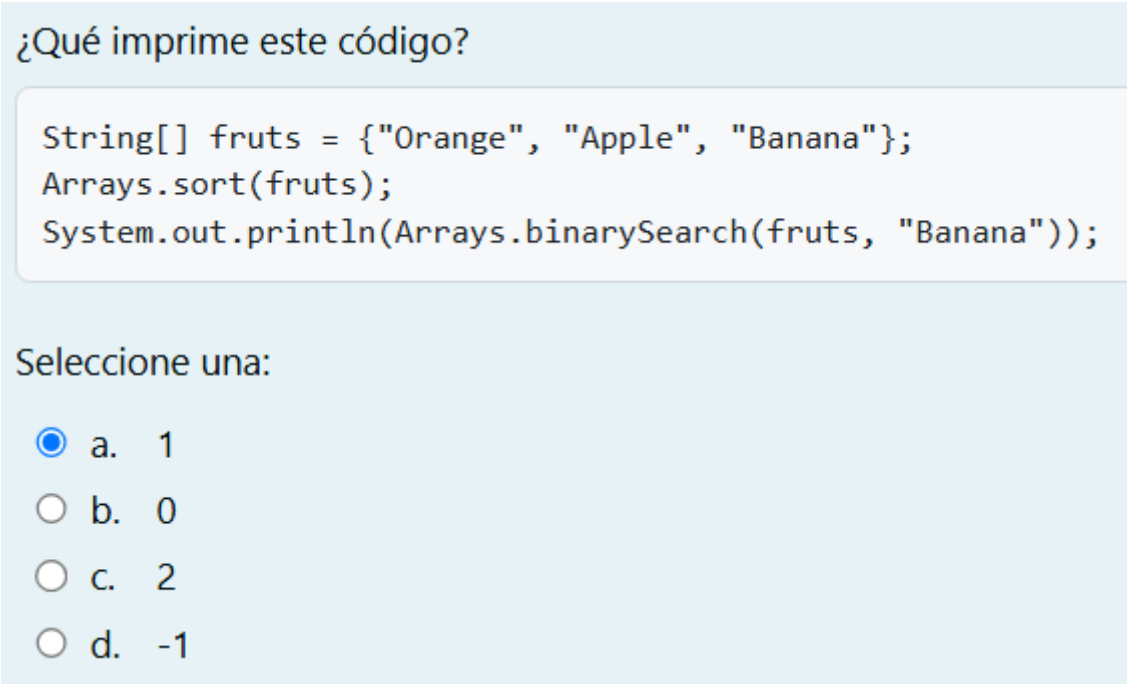
Seleccione una:

- a. 3false3
- b. 3true1
- c. 3false1
- d. 2false1
- e. 2false3
- f. 2true3
- g. 2true1

Respuesta: En este ejercicio estamos evaluando las propiedades de los arreglos bidimensionales, que en Java son esencialmente "arreglos de arreglos", además de ver cómo el lenguaje trata a los arreglos internamente como objetos. Hay que desglosar la ejecución línea por línea para entender lo que está pasando. Comenzamos con array2D[0].length: Se accede a la primera fila de la matriz, que es el arreglo interno {0, 1, 2}. Al consultar su propiedad length, el resultado es 3. Como se utiliza el método print (y no println), el 3 se imprime en la consola y el cursor se mantiene en la misma línea. Seguimos con array2D[1].getClass().isArray(): Se accede a la segunda fila, que es el arreglo {3, 4, 5, 6}. Al invocar .getClass() se obtiene el objeto Class representativo en tiempo de ejecución y el método isArray() confirma correctamente que esa clase corresponde a un arreglo, devolviendo el

booleano true. Se concatena en la consola. Terminamos con array2D[0][1]: Se accede nuevamente a la primera fila {0, 1, 2} y se solicita el elemento en el índice 1, el cual es el número 1. Al ejecutarse secuencialmente sin saltos de línea intermedios, la salida combinada es exactamente 3true1.

Pregunta número 6



Respuesta: En este ejercicio se está evaluando el uso de los métodos de la clase utilitaria Arrays sobre un arreglo de cadenas String. Hay que partir del estado inicial: Primero se declara un arreglo fruts con los elementos {"Orange", "Apple", "Banana"}. De ahí continuamos con el ordenamiento: En la instrucción Arrays.sort(fruts), se reorganizan los elementos del arreglo en estricto orden alfabético. El arreglo en memoria pasa a ser {"Apple", "Banana", "Orange"}. Es importante recordar que binarySearch exige que la estructura esté ordenada previamente para garantizar resultados predecibles. Continuamos con la búsqueda: La instrucción Arrays.binarySearch(fruts, "Banana") busca la cadena exacta "Banana" dentro de este nuevo arreglo ordenado. Finalmente, llegamos a la resolución de índices: En la nueva disposición, "Apple" se encuentra en el índice 0, "Banana" se encuentra en el índice 1 y "Orange" ocupa el índice 2. Al localizar el elemento, el método devuelve su posición actual. Como "Banana" se encuentra en el índice 1, el método devuelve 1.

Pregunta número 7

¿Que imprime en pantalla el código anterior?

```
import java.util.Arrays;
public class Test {
    public static void main(String[] args) {
        String[] strings = { "2", "3", "1", "4" };
        Arrays.sort(strings, 1, 4);
        System.out.println(Arrays.toString(strings));
    }
}
```

Seleccione una:

- a. [2, 3, 1, 4]
- b. [2, 1, 3, 4]
- c. [1, 2, 3, 4]
- d. [2, 4, 1, 3]
- e. [4, 3, 2, 1]

Respuesta: En este código se está evaluando el uso del método sobrecargado Arrays.sort(), que permite ordenar solo un subrango específico de un arreglo. Hay que desglosar el comportamiento del método Arrays.sort(strings, 1, 4) paso a paso para entender su comportamiento: En el estado inicial, el arreglo se inicializa como {"2", "3", "1", "4"}. Después, pasando a la definición de límites, en la API de Java, cuando se define un rango con índices, el parámetro fromIndex (1) es inclusivo, mientras que el toIndex (4) es exclusivo. Continuando con la extracción del subrango, tenemos que el método seleccionará únicamente los elementos ubicados en los índices 1, 2 y 3, los cuales corresponden a "3", "1" y "4". El elemento en el índice 0 ("2") queda excluido y no sufrirá ninguna modificación. Por otro lado, en el ordenamiento, el subrango se ordena de forma natural, pasando de "3", "1", "4" a "1", "3", "4". Siguiendo con la reconstrucción, al combinar el elemento intacto en la posición 0 con el subrango ordenado, el arreglo final en memoria resulta en {"2", "1", "3", "4"}. Finalmente, Arrays.toString(strings) formatea la salida con los corchetes estándar, dando como salida: [2, 1, 3, 4].

Pregunta número 8

What is the result?

```
public static void main(String[]args){
    int[] array = { 1,2,3,4,5};
    System.arraycopy(array, 2, array, 1, 2);
    System.out.print(array[1]);
    System.out.print(array[4]);
}
```

Seleccione una:

- a. 14
- b. 15
- c. 34
- d. 24
- e. 35
- f. 25

Respuesta: Aquí se está evaluando el comportamiento del método nativo System.arraycopy(), el cual se utiliza para copiar de forma eficiente porciones de un arreglo. Lo primero que debemos tener presente es que su firma recibe cinco parámetros: arreglo origen, posición origen, arreglo destino, posición destino y cantidad de elementos a copiar. En este caso, el arreglo está leyendo y escribiendo sobre sí mismo: Nuestro estado inicial es el arreglo {1, 2, 3, 4, 5} con índices del 0 al 4. Continuamos con System.arraycopy(array, 2, array, 1, 2): Toma como origen el mismo array, partiendo del índice 2 (el valor 3). La cantidad de elementos a copiar es 2, por lo que tomará los valores 3 y 4 (índices 2 y 3 originales). El destino es el mismo array, partiendo del índice 1. Al reescribir, el índice 1 pasa de 2 a 3, y el índice 2 pasa de 3 a 4. Con esto llegamos al estado final: El arreglo en memoria queda como {1, 3, 4, 4, 5}. El System.out.print(array[1]); imprime el nuevo valor en esa posición, que es 3, mientras que System.out.print(array[4]); imprime el valor en la última posición, que se mantuvo intacto como 5. Al ejecutarse de forma secuencial sin saltos de línea, la salida por consola es 35.

Pregunta número 9

Which two are valid instantiations and initializations of a multidimensional array?

Seleccione una o más de una:

- [x] a. int[][] array2D = new int[2][2]; array2D[0][0] = 1; array2D[0][1] = 2; array2D[1][0] = 3; array2D[1][1] = 4;
- [x] b. int[] array = {0, 1}; int[][][] array3D = new int[2][2][2]; array3D[0][0] = array; array3D[0][1] = array; array3D[1][0] = array; array3D[0][1] = array;
- [ ] c. int[][] array2D = {0, 1};
- [ ] d. int[][][] array3D = {{0, 1}, {2, 3}, {4, 5}};
- [ ] e. int[][] array2D = {{0, 1, 2, 4}{5, 6}};

Respuesta: En este ejercicio se está evaluando la comprensión de que, en Java, los arreglos multidimensionales son estructuralmente "arreglos de arreglos", lo que permite manipulaciones directas sobre sus referencias internas. La primera opción válida es el inciso a, ya que instancia explícitamente una matriz de 2x2 en memoria usando new int[2][2] y luego procede a inicializar cada una de sus celdas individuales (que almacenan tipos primitivos int) mediante sus coordenadas exactas. La otra opción válida es el inciso b. Primero crea un arreglo unidimensional array de tipo int[]. Luego instancia un arreglo tridimensional array3D de tipo int[][][]. Como un arreglo 3D contiene arreglos 2D, y estos a su vez contienen arreglos 1D, la expresión array3D[0][0] apunta directamente a un espacio diseñado para almacenar una referencia de tipo int[]. Por lo tanto, asignarle directamente la referencia de nuestro primer array es perfectamente legal a nivel de compilación y ejecución.